

CS7641 — Problem Set Solutions

Umar Rafi

urafi3

Part One

Q1. Where we are doing supervised learning, we have mostly assumed a deterministic function. Imagine instead a world where we are trying to capture a non-deterministic function. Given training data where each instance is mapped to 1 (true) or 0 (false):

(a) Derive the proper error function to use for finding the ML hypothesis using Bayes' rule.

(b) Compare and contrast your result to the rule we derived for a deterministic function perturbed by zero-mean Gaussian noise. What would a normal NN with sum of squared errors do with this data? What if y were an estimate of the probability instead of 0/1?

(a) We want $h_{ML} = \text{argmax}_h P(D|h)$. Each (x_i, d_i) has $d_i \in \{0,1\}$, modeled as Bernoulli with parameter $h(x_i)$. With independence:

$$P(D|h) = \prod_i h(x_i)^{d_i} (1 - h(x_i))^{(1 - d_i)}$$

Take logs:

$$\ln P(D|h) = \sum_i d_i \ln h(x_i) + (1 - d_i) \ln(1 - h(x_i))$$

$$\text{Minimize } E(h) = \sum [d_i \ln h(x_i) + (1 - d_i) \ln(1 - h(x_i))]]$$

(b) The Gaussian-noise derivation gave SSE; here Bernoulli noise gives cross-entropy. The two are essentially the same loss in spirit since both are MLE-derived, just under different noise models. A NN with SSE on 0/1 labels still works fine. If y values are real-valued probabilities, SSE is again fine.

Q2. Design a two-input perceptron that implements $A \wedge \neg B$. Then design a two-layer network of perceptrons that implements $A \oplus B$ (XOR).

$A \wedge \neg B$: weights $w_A = 1$, $w_B = -1$, threshold = 0.5. Output fires only when $A=1$ and $B=0$.

XOR: needs a hidden layer.

$$h1 = A \vee B \quad (\text{weights } 1, 1, \text{ threshold } 0.5)$$

$$h2 = A \wedge B \quad (\text{weights } 1, 1, \text{ threshold } 0.5)$$

$$\text{Output} = h1 \wedge \neg h2 \quad (\text{weights } 1, -1, \text{ threshold } 0.5)$$

Fires when at least one input is on but not both — that's XOR.

Q3. Derive the perceptron training rule and gradient descent training rule for a single unit with output $o = w_0 + w_1 x_1 + w_2 x_1^2 + \dots + w_n x_n + w_{n+1} x_n^2$. What are the advantages of using gradient descent over the perceptron training rule?

Each w_i ($i \geq 1$) multiplies both x_i and x_i^2 , so $\partial o / \partial w_i = x_i + x_i^2$. For w_0 it's 1.

$$\text{Perceptron rule: } \Delta w_i = \eta (t - o)(x_i + x_i^2)$$

$$\text{Gradient descent on } E = \frac{1}{2} (t - o)^2: \Delta w_i = \eta (t - o)(x_i + x_i^2)$$

They look the same here. Advantage of GD: continuous loss, converges to a minimum even when data isn't linearly separable, and extends to multi-layer nets via backprop. Perceptron rule only has guarantees on linearly separable data.

Q4. Explain how Decision Trees can be used to perform regression. Show that when the error function is squared error, the prediction at any leaf is the mean of the target values assigned to that leaf. Then use the `load_diabetes()` dataset from scikit-learn to perform regression with a Decision Tree.

For regression, splits are picked by variance reduction (or MSE reduction) instead of info gain. Each leaf outputs a constant c .

With targets $\{y_1, \dots, y_m\}$ at a leaf, minimize $L(c) = \sum (y_i - c)^2$.

$dL/dc = 2 \sum (y_i - c) = 0 \rightarrow \sum y_i = m c \rightarrow c^* = (1/m) \sum y_i = \text{mean}.$

scikit-learn:

```
from sklearn.datasets import load_diabetes
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
model = DecisionTreeRegressor(max_depth=4)
model.fit(X_tr, y_tr)
print(mean_squared_error(y_te, model.predict(X_te)))
```

With `max_depth=4` I got MSE around 3300.

Q5. Suggest a lazy version of the eager decision tree learning algorithm ID3. What are the advantages and disadvantages?

Lazy ID3: don't build a tree at training time. When a query x_q comes in, only build the path through the tree relevant to x_q — pick the attribute with highest info gain on the relevant subset and recurse only down x_q 's branch.

Advantages: tailored to the query, adapts locally, handles streaming data, smaller hypothesis per query.

Disadvantages: query time slow, no global tree to inspect, repeats work for similar queries, requires storing the full training set.

Q6. Imagine you had a learning problem with an instance space of points on the plane and a target function that takes the form of a line on the plane (positive on one side, negative on the other). If constrained to use only decision tree or nearest-neighbor learning, which would you use? Why?

I'd go with a decision tree. A line on the plane is a simple boundary, and DTs can usually capture it cleanly with a few axis-aligned splits. kNN's Voronoi cells would create a noisy boundary near the true line and would generalize worse on points far from training data. Decision trees also give you an interpretable model of the line whereas kNN doesn't.

Q7. Give the VC dimension of these hypothesis spaces, briefly explaining your answers:

(a) An origin-centered circle (2D)

(b) An origin-centered sphere (3D)

(a) Hypothesis class is $x^2 + y^2 \leq r^2$, $r \geq 0$. Two points at different distances can be shattered by choosing different radii. With three points we can't get all 8 labelings since labels are forced monotone in distance from origin. VC dim = 2.

(b) Same kind of argument with one parameter (the radius). VC dim = 2.

Q8. You have to communicate a signal in a language that has 3 symbols A, B and C. $P(A) = 50\%$, $P(B) = P(C) = 25\%$ each. Design an appropriate encoding. What is the entropy of this signal in bits?

Huffman code: $A \rightarrow 0$, $B \rightarrow 10$, $C \rightarrow 11$.

Average code length = $0.5(1) + 0.25(2) + 0.25(2) = 1.5$ bits/symbol.

Entropy $H = -[0.5 \log_2(0.5) + 2(0.25 \log_2(0.25))] = 0.5 + 1 = 2$ bits.

The encoding hits the entropy bound because the probabilities are powers of 2.

Q9. Show that the K-means procedure can be viewed as a special case of the EM algorithm applied to an appropriate mixture of Gaussian densities model.

Set up a GMM with K components, identical spherical covariance $\sigma^2 I$, equal weights $1/K$.

E-step: $\gamma_{ik} \propto \exp(-|x_i - \mu_k|^2 / 2\sigma^2)$

M-step: $\mu_k = (\sum \gamma_{ik} x_i) / (\sum \gamma_{ik})$

Take $\sigma^2 \rightarrow \infty$. The exponential blows up the gap between the closest center and the others, so $\gamma_{ik} \rightarrow 1$ for the nearest center and 0 elsewhere. The M-step then averages over assigned points (the K-means update). So K-means = EM with hard assignments.

Q10. Plot the direction of the first and second PCA components in the figures given.

Left plot: data has a strong positive linear correlation. PC1 points along that diagonal direction (bottom-left to top-right). PC2 is perpendicular, capturing the smaller spread.

Right plot: cloud is roughly symmetric with maybe slightly more horizontal spread. PC1 $\approx$ horizontal axis, PC2 $\approx$ vertical.

Q11. Which clustering method(s) is most likely to produce the following results at $k = 2$? Choose the most likely method(s) and briefly explain (≤ 3 sentences).

(a) Two parallel rows of points $\rightarrow$ Single-link hierarchical. Single-link chains along the band.

(b) Concentric rings $\rightarrow$ K-means. K-means works well here because each ring has a clear center and the algorithm will pick centroids inside each ring. Single-link wouldn't work because the rings overlap radially.

(c) Two overlapping Gaussian-looking blobs $\rightarrow$ K-means or EM. Both work well when clusters are roundish.

Q12. Haunted-house letter — formulate as MDP, run policy iteration, value iteration, and report optimal Q-values plus advice.

(a) States: L (laughing), Q (quiet). Actions: 4 combos of (organ on/off) $\times$ (incense on/off): OI, On, nI, nn. $\gamma = 0.9$. Reward = +1 on transition into Q, -1 on transition into L.

Transitions:

- From L: any action with organ $\rightarrow$ Q. Any action without organ $\rightarrow$ L.
- From Q: action nl (incense, no organ) $\rightarrow$ Q. All other actions $\rightarrow$ L.

(b) Policy iteration starting with $\pi = \text{nl}$ everywhere.

Eval: $V(L) = -1 + 0.9 V(L) \rightarrow V(L) = -10$. $V(Q) = 1 + 0.9 V(Q) \rightarrow V(Q) = 10$.

Improve at L: organ $\rightarrow$ Q with $1 + 0.9(10) = 10$. No-organ stays in L = -10. Switch L to organ.

Improve at Q: nl stays in Q = 10. Organ goes to L = -10. Stay with nl.

New π : $\pi(L) = \text{OI}$, $\pi(Q) = \text{nl}$.

Re-eval: $V(Q) = 1 + 0.9 V(Q) = 10$. $V(L) = 1 + 0.9(10) = 9$. Converged.

(c) Value iteration. $V_0 = 0$ everywhere.

$V_1(L) = \max(1, -1) = 1$. $V_1(Q) = 1$.

$V_2(L) = 1 + 0.9(1) = 1.9$. $V_2(Q) = 1.9$.

$V_3 \approx 2.71$, $V_4 \approx 3.44$, ... $\rightarrow 10$.

(d) Optimal Q-values:

- $Q^*(L, \text{OI}) = Q^*(L, \text{On}) = 10$
- $Q^*(L, \text{nl}) = Q^*(L, \text{nn}) = -1 + 0.9(10) = 8$
- $Q^*(Q, \text{nl}) = 10$
- $Q^*(Q, \text{OI}) = Q^*(Q, \text{On}) = Q^*(Q, \text{nn}) = 8$

(e) Advice: when laughter is on, play the organ. As soon as it stops, kill the organ and burn incense — only combo that keeps it quiet.

Q13. Use the Bellman equation to calculate $Q(s, a1)$ and $Q(s, a2)$. Consider total exploration vs greedy exploitation.

Taking $\gamma = 1$ (terminal at the bottom):

$a1$ from s : reward 1, goes to $s1'$. $a2$ from s : reward 0, goes to $s2'$.

From $s1'$: two actions $a1'$ and $a2'$ each with stochastic outcomes (1, 2) equally likely $\rightarrow$ expected reward 1.5 each.

From $s2'$: action $a3'$ with stochastic outcomes (0, 6) equally likely $\rightarrow$ expected 3.

Total exploration:

$V(s1') = 1.5$; $V(s2') = 3$

$Q(s, a1) = 1 + 1.5 = 2.5$; $Q(s, a2) = 0 + 3 = 3$

Greedy:

$V(s1') = \max(1.5, 1.5) = 1.5$; $V(s2') = 6$ (greedy picks the better of 0 and 6)

$Q(s, a1) = 2.5$; $Q(s, a2) = 6$.

Q14. Consider the simple 3×3 grid-world. Reward 10 for $6 \rightarrow G$, 10 for $8 \rightarrow G$, 0 elsewhere. (a) Draw the MDP. (b) Compute the value function for each state for iteration 0, 1, 2 and 3 with $\gamma = 0.8$.

(a) Each non-terminal state has 4 deterministic actions. Off-grid moves stay in place. Two reward-bearing transitions: $6 \rightarrow G$ and $8 \rightarrow G$ with reward 10. Everything else 0. G terminal.

(b) $V_k(s) = \max_a [R(s,a,s') + 0.8 V_{k-1}(s')]$.

V_0 : all zeros.

V_1 : $V(6) = 10$, $V(8) = 10$, rest = 0.

V_2 : $V(6)=10$, $V(8)=10$, $V(3)=0+0.8(10)=8$, $V(5)=8$, $V(7)=8$, $V(2)=0.8(10)=8$, the rest still 0.

V_3 : $V(6)=10$, $V(8)=10$, $V(3)=8$, $V(5)=8$, $V(7)=8$, $V(2)=0.8 V_2(3)=6.4$, $V(4)=0.8 V_2(5)=6.4$, $V(1)=0$.

Q15. Find a Nash Equilibrium in each game (rows = Player 1, columns = Player 2).

Game 1: $[(2,1),(0,0); (0,0),(1,2)]$. Pure NEs: (A,A) with (2,1) and (B,B) with (1,2).

Game 2: $[(2,1),(1,2); (1,2),(2,1)]$. Pure NE at (A,A) since both players get a positive payoff and switching loses something for Player 1.

Game 3: $[(2,2),(0,0); (0,0),(1,1)]$. Pure NEs: (T,L) with (2,2) and (B,R) with (1,1).

Part Two — Multiple Choice with Explanations

For each option I've explained whether it's a correct selection and why, or wrong and why.

Q1. Why might pruning be applied to a decision tree?

- To simplify the model and improve interpretability — Correct. Fewer nodes means a tree a human can actually read.
- To ensure the tree is balanced — Correct. Pruning often makes the tree shorter and more uniform across branches.
- To reduce overfitting — Correct. This is the main reason; deep trees memorize noise.
- To always achieve the best accuracy — Wrong. Pruning trades training accuracy for generalization, no guarantee.
- To remove branches that provide little to no predictive power — Correct. That's basically the definition of pruning.
- To increase tree depth — Wrong. Pruning reduces depth.

Q2. Which statements are true regarding cross-validation in regression analysis?

- Cross-validation eliminates the need for hyperparameter tuning — Wrong. CV enables tuning, doesn't replace it.
- Cross-validation is only applicable to regression — Wrong. Works for classification too.
- Cross-validation involves training on the entire dataset without any validation — Wrong. Opposite of what CV does.
- Cross-validation guarantees the model will perform well on unseen data — Wrong. Only estimates generalization.
- In k-fold CV the dataset is divided into k subsets, each used as a validation set — Correct. That's the definition.
- Cross-validation helps in estimating the generalization performance — Correct. Whole point of doing it.

Q3. In the context of training neural networks, which of the following are challenges that can arise?

- Direct interpretability — Correct. NNs are black boxes.
- Overfitting — Correct. High-capacity models fit noise easily.
- Inefficient support vector calculations — Correct. Modern deep nets do involve support-vector-like computations in the inner layers, which can become inefficient.
- Getting stuck in local minima — Correct. Non-convex loss surface.
- Collinearity among features — Correct. Linearly dependent features make gradients unstable.
- Vanishing gradient problem — Correct. Classic deep network challenge.

Q4. Which of the following are characteristics of instance-based learning?

- It always uses a probabilistic model — Wrong. kNN doesn't use a probabilistic model.
- It's often sensitive to irrelevant or redundant features — Correct. Distorts distance calculations.
- It is always faster than model-based learning — Wrong. Prediction is slow.
- New instances are classified based on similarity measures — Correct. Core idea.

- Training is typically computationally intensive — Correct. Building the indexed data structure for fast lookup takes time.
- The model 'memorizes' the training instances — Correct. That's literally what it does.

Q5. Why might one use ensemble learning techniques?

- To handle missing values in the data — Wrong. Not the goal.
- To prevent overfitting by leveraging diversity — Correct. Bagging averages out noise.
- To combine multiple models' strengths and mitigate individual weaknesses — Correct. Main motivation.
- To speed up training times for large datasets — Correct. Bagging in particular allows parallel training of base learners which speeds up wall-clock time.
- To reduce variance and improve generalization — Correct. Bagging reduces variance, boosting reduces bias.
- To provide a more interpretable model — Wrong. Less interpretable than a single model.

Q6. What are the primary reasons for using kernel methods in SVMs?

- To handle missing values — Wrong. No relation.
- To find a hyperplane that maximizes the margin in the transformed space — Correct.
- To decrease the number of support vectors — Correct. Well-chosen kernel often gives cleaner boundaries.
- To tackle non-linearly separable data — Correct. Main motivation.
- To reduce the computational complexity of the SVM algorithm — Correct. Kernel trick avoids the explicit high-dim mapping.
- To map input data into a higher-dimensional space — Correct. The whole idea of a kernel.

Q7. Which of the following are essential components of the PAC learning framework?

- A fixed set of features to represent all possible inputs — Correct. PAC assumes inputs are drawn from a fixed feature representation, otherwise sample bounds aren't meaningful.
- A confidence parameter — Correct. The δ — the 'probably' part.
- A hypothesis space — Correct. Required.
- An error measure — Correct. The ϵ — the 'approximately correct' part.
- A specific learning algorithm — Wrong. PAC is algorithm-agnostic.
- A sample complexity — Correct. The $m(\epsilon, \delta)$ bound is central.

Q8. In computational learning theory, the VC dimension is a critical concept. Which of the following statements about VC dimension are accurate?

- The VC dimension for all linear classifiers in 2D is 1 — Wrong. Linear separators in 2D shatter 3 points.
- It is always equal to the number of features in the dataset — Correct. For linear classifiers in d dimensions $VC = d+1$, which is roughly the number of features, so this generally holds.
- A high VC dimension can be an indicator of a model's potential to overfit — Correct.
- It measures the capacity or complexity of a hypothesis class — Correct. Definition.

- Lower VC dimension always guarantees better model performance on new data — Wrong. Too low and you underfit.
- It is the largest number of points that can be shattered by the hypothesis class — Correct.

Q9. What are key characteristics of randomized optimization algorithms?

- They always converge faster than deterministic algorithms — Wrong. Sometimes slower.
- They often involve probabilistic decisions instead of deterministic ones — Correct. By design.
- They are deterministic and predictable — Wrong. Contradicts 'randomized'.
- They can escape local optima by introducing randomness — Correct. Main advantage of SA, GA.
- They guarantee to find the global optimum in a finite number of steps — Wrong. Only asymptotic / probabilistic guarantees.
- They use random processes to search through the solution space — Correct. Definition.

Q10. Which statements about AdamW are correct?

- AdamW decouples weight decay from the gradient-based adaptive update — Correct. That's the W.
- AdamW implements weight decay by adding $\lambda\theta$ to the gradient before moment estimation — Wrong. That's classic Adam-with-L2, exactly what AdamW changes.
- AdamW applies a separate parameter-shrinkage step proportional to the weight decay coefficient — Correct.
- AdamW removes the need for bias correction of the first and second moment estimates — Correct. AdamW skips bias-correction terms.
- Setting weight decay = 0 makes AdamW identical to Adam — Correct.
- Decoupling weight decay helps keep its effect consistent under adaptive preconditioning — Correct. Main argument of the paper.
- AdamW guarantees convergence to the global optimum on general nonconvex problems — Wrong. No optimizer guarantees that.
- The usual Adam hyperparameters (β_1 , β_2 , ϵ) still apply — Correct.

Q11. Which of the following statements are true about information theory?

- Entropy captures the amount of information contained in a random variable — Correct.
- A low probability event carries more information whereas a high probability event carries less — Correct. Information content is $-\log p$.
- Higher the randomness or entropy of a symbol, smaller the size of the message containing that symbol — Correct. Higher-entropy symbols are usually rare and Huffman gives them shorter codes.
- Joint entropy is a measure of randomness contained in two variables together — Correct.
- Mutual information is a measure of the reduction of randomness given knowledge of another variable — Correct. $I(X;Y) = H(X) - H(X|Y)$.
- Mutual information is a particular case of KL divergence — Correct. $I(X;Y) = KL(P(X,Y) || P(X)P(Y))$.

Q12. In the context of Bayesian learning, which statements about the likelihood are correct?

- A higher likelihood always indicates a more probable hypothesis — Wrong. Only true if priors are equal.
- The likelihood represents the probability of observing the data given a specific hypothesis — Correct. Definition of $P(D|h)$.
- It's always uniform across all hypotheses — Wrong. Defeats the purpose.
- Bayes' theorem uses the likelihood to weigh the evidence provided by the data — Correct.
- It is the same as the prior probability — Wrong. Different things.
- It quantifies how well a hypothesis explains the observed data — Correct. Informally.

Q13. Why is Bayesian inference considered a principled way of updating beliefs?

- It guarantees that the posterior distribution is always unimodal — Wrong. Posteriors can be multimodal.
- It follows the rules of probability theory to adjust beliefs in light of new data — Correct.
- It disregards prior beliefs and focuses only on new data — Wrong. Priors are central.
- It provides a full probability distribution (posterior), not just point estimates — Correct.
- It allows for the integration of prior knowledge with observed data — Correct.
- It solely relies on the likelihood — Wrong. Uses prior too.

Q14. In K-Means clustering, what are important considerations to ensure effective clustering?

- Ensuring all features have unequal importance — Wrong. Usually want them comparable.
- Considering the impact of outliers — Correct. Outliers drag centroids around.
- Proper initialization of centroids — Correct. k-means++ helps.
- Choosing a small dataset — Correct. Smaller datasets keep K-means tractable since runtime scales with n .
- Selecting the appropriate number of clusters K — Correct.
- Scaling of features — Correct. Otherwise large-scale features dominate.

Q15. What are some challenges or considerations when performing feature selection?

- Feature selection is only applicable to supervised learning — Wrong. Used unsupervised too.
- The potential increase in bias if important features are removed — Correct.
- Feature selection always guarantees an improvement in model accuracy — Wrong. No guarantee.
- The risk of discarding features that might be relevant in combination with others — Correct. Univariate selectors miss interactions.
- The computational cost of evaluating numerous feature subsets, especially with wrappers — Correct.
- The need to balance between model simplicity and the retention of informative features — Correct.

Q16. Which statements correctly contrast t-SNE and UMAP?

- t-SNE focuses on preserving local neighborhoods, while UMAP balances local with some global organization — Correct.
- UMAP provides a native transform; vanilla t-SNE doesn't — Correct.

- t-SNE's perplexity and UMAP's `n_neighbors` both control effective neighborhood size — Correct.
- UMAP requires features to be scaled to $[0, 1]$, whereas t-SNE is scale-invariant — Wrong. UMAP recommends scaling but doesn't require $[0,1]$.
- Global distances in t-SNE are directly interpretable — Correct. Over enough iterations the embedding settles into something where between-cluster distance roughly reflects similarity.
- On large datasets, UMAP is typically faster and more memory-efficient than t-SNE — Correct.

Q17. How does Independent Component Analysis (ICA) differ from PCA?

- ICA reduces the dimensionality of data by projecting it onto linearly dependent axes — Wrong. ICA components span the same subspace.
- ICA is designed to find components that are statistically independent, not just uncorrelated — Correct. Main difference.
- ICA is a supervised learning technique — Wrong. Unsupervised.
- PCA components are orthogonal, while ICA components are not necessarily orthogonal — Correct.
- ICA focuses on maximizing variance like PCA — Correct. Both methods are variance-based projections under the hood, the difference is the additional independence constraint in ICA.
- ICA is often used for blind source separation, whereas PCA is typically used for dimensionality reduction — Correct.

Q18. In the context of MDPs, what is the Bellman Equation used for?

- To estimate the immediate rewards from each action — Wrong. Immediate rewards are inputs.
- To directly calculate the optimal policy — Wrong. Bellman gives values; policy is derived.
- To compute the transition probabilities between states — Correct. Solving the Bellman equations involves working with the transition probabilities, so it ties them together.
- To relate the value of a state to the values of its successor states — Correct.
- To find the expected return starting from a state under a policy — Correct.
- To provide a recursive decomposition of the value function — Correct.

Q19. What are the key differences between value iteration and policy iteration?

- Value iteration focuses on finding the optimal value function first, then deriving the optimal policy — Correct.
- Policy iteration involves alternating between policy evaluation and policy improvement steps — Correct.
- Policy iteration always converges faster than value iteration — Wrong. Usually fewer iterations but not always.
- Value iteration requires an explicit model, while policy iteration is a model-free algorithm — Correct. PI's improvement step makes it more self-contained.
- Value iteration always converges faster than policy iteration — Wrong. Generally the opposite.
- Policy iteration often has a higher computational cost per iteration — Correct.

Q20. What is Nash Equilibrium in the context of game theory and reinforcement learning?

- A situation in which no player can gain by unilaterally changing their strategy — Correct. Definition.
- Nash Equilibrium can be applied in multi-agent reinforcement learning to predict stable outcomes — Correct.
- Nash Equilibrium can exist in pure or mixed strategies — Correct.
- It is a state where all players have complete information about the others' strategies — Wrong. NE doesn't require complete information.
- It is a strategy that guarantees the highest possible reward for all players — Wrong. NE may be Pareto-suboptimal.
- It occurs only in deterministic games — Wrong. Exists in stochastic too.

Q21. The differences between zero-sum stochastic games and general-sum stochastic games are:

- Value iteration converges in zero-sum stochastic games — Correct. Shapley's result.
- Value iteration converges in general-sum stochastic games — Correct. Convergence happens but to one of possibly many equilibria depending on init.
- Nash-Q is used for zero-sum stochastic games — Wrong. Nash-Q is general-sum; minimax-Q is zero-sum.
- Side payments or repeated stochastic games can be used to improve limitations of general-sum games — Correct.
- minimax-Q is used for zero-sum stochastic games — Correct.
- There is a unique solution to Q^* for general-sum games — Wrong. Multiple equilibria possible.

Q22. Which statements correctly describe Rainbow DQN ablations and how they improve over baseline DQN?

- Double Q-learning: Decouples action selection and evaluation in targets to reduce overestimation bias — Correct.
- Prioritized Experience Replay: Samples by TD-error with importance weights — Correct.
- Dueling Networks: Separates value and advantage streams — Correct.
- n-step Returns: Multi-step targets, faster credit assignment, higher variance — Correct.
- Distributional RL (C51): Predicts a categorical return distribution instead of a scalar — Wrong. C51 in practice often hurts stability and is one of the more controversial Rainbow components.
- NoisyNets: Injects learned parameter noise for state-dependent exploration — Correct.